1. Aim of the Experiment

To build and train a Convolutional Neural Network (CNN) for image classification using the CIFAR-10 dataset, apply basic data augmentation techniques, and compare its performance with a Multilayer Perceptron (MLP) based on test accuracy, test loss, and training time.

2. Theory

Image classification is a computer vision task in which a model identifies and assigns a label to an image. Deep learning models learn patterns from images, such as edges, textures, shapes, and object features.

Convolutional Neural Network (CNN)

A CNN is a deep learning architecture designed to process grid-like data such as images. It uses convolutional layers to extract spatial features, activation functions to introduce non-linearity, and pooling layers to reduce spatial dimensions.

- Convolutional layer: Applies learnable filters to extract features such as edges, textures, and patterns.
- ReLU activation: Introduces non-linearity by setting negative activations to zero.
- Max pooling: Reduces feature-map dimensions while retaining prominent features.
- Flatten layer: Converts multidimensional feature maps into a one-dimensional vector.
- Fully connected layer: Learns relationships between extracted features for classification.
- Softmax layer: Produces class probabilities for the ten CIFAR-10 categories.

Multilayer Perceptron (MLP)

An MLP is a feedforward neural network consisting of fully connected layers. Each image is flattened into a one-dimensional vector before being passed through dense layers. Unlike a CNN, the MLP does not explicitly preserve spatial relationships between neighboring pixels.

Data Augmentation

Data augmentation artificially increases the variety of training examples. The CNN setup described for this experiment uses random horizontal flipping, random rotation of up to 10%, and random zoom of up to 10%. These transformations can help the model generalize to variations in image orientation and scale.

3. Dataset Used

CIFAR-10 is a labeled image dataset containing 60,000 color images divided into ten classes. Each image has a resolution of 32 × 32 pixels and three color channels (RGB).

Dataset property	Description
Total images	60,000
Training images	50,000
Testing images	10,000
Image dimensions	32 × 32 × 3
Number of classes	10
Training/validation split	90% / 10%
Image normalization	Pixel values divided by 255

The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The notebook loads CIFAR-10 using TensorFlow/Keras, normalizes pixel values to the range 0–1, and uses 10% of the training data for validation.

4. Architecture of the Models

A. Multilayer Perceptron (MLP)

The MLP takes a 32 × 32 × 3 image as input and flattens it into a vector of 3,072 values.

Layer	Configuration
Input	32 × 32 × 3
Flatten	3,072 features
Dense + ReLU	512 neurons
Dropout	30%
Dense + ReLU	256 neurons
Dropout	30%
Output + Softmax	10 classes

B. Convolutional Neural Network (CNN)

The CNN processes the original image while preserving its spatial structure. The architecture is summarized below.

Layer	Configuration
Input image	32 × 32 × 3
Data augmentation	Random horizontal flip, rotation, zoom
Conv2D + ReLU	32 filters, 3 × 3, same padding
Max pooling	2 × 2
Conv2D + ReLU	64 filters, 3 × 3, same padding
Max pooling	2 × 2
Flatten	8 × 8 × 64 = 4,096 features

Dense + ReLU	128 neurons
Dropout	40%
Output + Softmax	10 classes

Common Training Configuration

Parameter	Value
Optimizer	Adam
Loss function	Sparse categorical cross-entropy
Evaluation metric	Accuracy
Epochs	10
Batch size	128
Validation split	10%
Framework	TensorFlow / Keras

5. Algorithm

1. Start the experiment and import TensorFlow, NumPy, Matplotlib, and time.
2. Load the CIFAR-10 training and testing datasets.
3. Normalize image pixel values by dividing them by 255.
4. Flatten the class labels and define the ten class names.
5. Display sample images from the training dataset.
6. Define data augmentation using random horizontal flipping, rotation, and zoom.
7. Build the MLP model with flattening, dense layers, dropout, and softmax classification.
8. Build the CNN model with augmentation, two convolutional layers, max pooling, a dense layer, dropout, and softmax classification.
9. Compile both models using Adam, sparse categorical cross-entropy, and accuracy.
10. Train the MLP for 10 epochs with batch size 128 and record the training time.
11. Train the CNN for 10 epochs using the same training configuration and record the training time.
12. Evaluate both models on the CIFAR-10 test dataset.
13. Compare test accuracy, test loss, and training time.
14. Plot training and validation accuracy and loss for both models.
15. Generate predictions for ten test images using the CNN and display actual and predicted classes.
16. Analyze the results and conclude the experiment.

6. Flowchart

Start → Import libraries → Load CIFAR-10 → Normalize data → Display samples → Define augmentation → Build MLP and CNN → Compile models → Train MLP and record time → Train CNN and record time → Evaluate both models → Compare metrics → Plot curves and predictions → Analyze results → End

Flow sequence: the dataset is prepared first; both models are then built and trained; their test results and learning curves are examined before drawing conclusions.

7. Analysis and Discussion

Feature extraction

The MLP directly processes flattened pixel values and learns through fully connected layers. The CNN extracts local spatial features through convolutional filters, allowing it to learn patterns such as edges, textures, and object parts.

Effect of data augmentation

The CNN uses random flipping, rotation, and zoom during training. These transformations expose the model to modified versions of training images and may improve generalization. The MLP does not use augmentation in the described setup, so the comparison reflects both architectural differences and augmentation.

Accuracy and loss

Test accuracy measures the proportion of correctly classified test images, while test loss measures prediction error using sparse categorical cross-entropy. Training and validation curves help identify convergence and possible overfitting.

Training time

Training time is the elapsed time for ten epochs and depends on the architecture and computing hardware. Convolution and augmentation add computation, while the MLP has a different dense-layer computation cost.

Prediction analysis

The notebook displays ten test images with actual and CNN-predicted labels. These examples illustrate individual successes and errors, but ten images are not enough to estimate overall performance.

8. Observation Table

Enter the measured outputs after executing the notebook. Numerical results are not provided in the notebook source, so they are left blank rather than estimated.

Sr. No.	Parameter	MLP	CNN
1	Test accuracy (%)	_____	_____
2	Test loss	_____	_____
3	Training time (seconds)	_____	_____
4	Number of epochs	10	10
5	Batch size	128	128
6	Data augmentation	No	Yes
7	Validation split	10%	10%

9. Conclusion

In this experiment, an MLP and a CNN were designed and trained for image classification using the CIFAR-10 dataset. The MLP used fully connected layers, whereas the CNN used convolutional and max-pooling layers to extract spatial features, together with data augmentation to introduce training-image variations.

Both models were evaluated using test accuracy, test loss, and training time. Their training and validation curves were plotted to study learning behavior, and CNN predictions were visualized on sample test images. The experiment demonstrates how two neural network architectures approach image classification differently and provides a practical basis for comparing their performance. The final conclusion about which model performed better should be made using the measured results in the observation table.